Programación defensiva

Docente : Héctor Xavier Limón Riaño
email: xavier120@hotmail.com

Introducción

Calidad en software

- La calidad se refiere al grado en que un producto satisface o cumple con los requerimientos del cliente
- Existe un problema de enfoque entre calidad en software y seguridad
- La seguridad suele estar relegada a segundo plano en materia de calidad

Calidad en software

- El software tiene tanto características de calidad internas y externas
- Las características externas son aquellas que el usuario del sistema puede percibir, incluyendo:
  - Usabilidad: la facilidad del usuario para aprender a usar el sistema
  - Eficiencia: uso mínimo de recursos, como memoria y procesador
  - Confiabilidad: es la habilidad del sistema para ejecutar sus funciones bajo diversas condiciones posibles produciendo la menor cantidad de errores

Calidad en software

Características externas de calidad (continuación):

- Integridad: grado en que el sistema previene el acceso no apropiado o no autorizado al sistema y a sus datos. Está también ligado a asegurar que los datos del usuario no se alteren de maneras impredecibles
- Adaptibilidad: la extensión en que un sistema puede ser utilizado sin modificarlo
- Certeza: el grado en que un sistema está libre de errores
- Robustez: es el grado en que el sistema continua funcionando ante la aparición de fallos o condiciones de estrés

Calidad en software

- Los programadores en cambio se preocupan por características internas del software así como por características externas
- Algunas características internas de calidad incluyen:
  - Mantenibilidad: facilidad de mantenimiento
  - Flexibilidad: que tanto puede modificarse un sistema para usos o ambientes diferentes para el que fue creado
  - Portabilidad
  - Reusabilidad
  - Legibilidad: que tan fácil es entender el código fuente
  - Facilidad de prueba
  - Facilidad de entendimiento: incluye tanto el bajo nivel como el alto nivel

Rutinas de alta calidad

Introducción

Una rutina puede ser:

- Una función (C++)
- Un método (Java)
- Un procedimiento (Pascal)
- Una macro (ensamblador)

Introducción

- **La seguridad en el código empieza teniendo buen código**
- Un código poco legible es difícil de auditar, incluyendo auditoría de seguridad
- Un código poco legible es difícil de corregir o extender
- Un mal código es más propenso a tener defectos de seguridad como mal manejo de memoria, condiciones de carrera o ciclos infinitos no intencionales

Rutinas de alta calidad

- Tienen un nombre descriptivo que no es demasiado corto ni demasiado largo, su nombre es un verbo
- No debe tener demasiados parámetros (aumenta el acoplamiento)
- Los parámetros deben seguir un orden lógico y consistente
- Los parámetros deben ser descriptivos
- No deberían introducir efectos colaterales
- Deben ser cohesivas (hacer sólo una cosa)
- Deben ser fáciles de leer y entender (siguen una estructura lógica)

Rutinas de alta calidad

- Tienen que estar bien documentadas
- No deben manipular memoria de forma descuidada
- Deben defenderse ante datos inválidos
- No utilizan números mágicos
- Utiliza todos los parámetros que recibe

Rutinas de alta calidad

```
void HandleStuff( CORP_DATA & inputRec, int crntQtr, EMP_DATA empRec, double
   & estimRevenue, double ytdRevenue, int screenX, int screenY, COLOR_TYPE &
   newColor, COLOR_TYPE & prevColor, StatusType & status, int expenseType )
{
int i;
for ( i = 0; i < 100; i++ ) {
   inputRec.revenue[i] = 0;
   inputRec.expense[i] = corpExpense[ crntQtr ][ i ];
   }
UpdateCorpDatabase( empRec );
estimRevenue = ytdRevenue * 4.0 / (double) crntQtr;
newColor = prevColor;
status = SUCCESS;
if ( expenseType == 1 ) {
     for ( i = 0; i < 12; i++ )
           profit[i] = revenue[i] - expense.type1[i];
     }
else if ( expenseType == 2 )  {
          profit[i] = revenue[i] - expense.type2[i];
          }
else if ( expenseType == 3 )
          profit[i] = revenue[i] - expense.type3[i];
```

Buenos nombres de rutinas

- Describe lo que una rutina hace
- No utiliza verbos genéricos de apoyo (hacer, ejecutar, etc.)
- Su nombre es tan largo como hace falta
- Es buen consejo apoyarse en lo que regresa la función para nombrarla
- Utiliza un verbo seguido de un sujeto
- Se apoyan del uso de opuestos: agregar/quitar, empezar/terminar, crear/destruir, primero/último, etc.
- Siguen convenciones en sus nombres (get, set, is)

Programación defensiva

Introducción

- Adopta la misma filosofía que al manejar un carro: nunca sabes lo que los otros conductores van a hacer mejor asume lo peor
- Te proteges a ti mismo aunque no sea tu culpa
- Si a una rutina se le pasan datos malos, no deberían de hacer daño, incluso si esos datos malos son producto de una rutina mal hecha

Introducción

- Aunque la programación defensiva está más orientada al tratamiento de bugs y manejo de errores esperados, también trata sobre seguridad
- En programación defensiva te proteges, por grado de importancia:
  - De ti mismo como programador (bugs)
  - De usuarios que comenten errores
  - De atacantes

Protegerse de datos de entrada inválidos

- Si recibes datos inválidos de ninguna forma se debe regresar algo, se debe generar un error
- Seguir estos consejos:
  - Revisar los valores de fuentes externas: revisar si están en un rango, un formato, etc. Ten especial cuidado ante posibles ataques (inyecciones, overflows, etc.)
  - Revisar los valores de todos los parámetros de rutinas: esencialmente igual al anterior
  - Decidir cómo manejar entradas malas: dependiendo de tipo de mala entrada se pueden hacer diversas cosas

Técnicas para el manejo de errores

- Para manejar errores que sabes pueden ocurrir
- Dependiendo de las circunstancias puede quererse hacer algo diferente, normalmente se lanza una excepción, se termina el programa, se utiliza un valor de retorno anterior que funcionó, se despliega un mensaje de error.

Excepciones

- Las excepciones son errores que se generan en tiempo de ejecución
- Se utilizan para avisar a otras partes del código acerca de errores que no deben ser ignorados. Las excepciones se propagan hacía arriba
- Lanza una excepción sólo para casos que sean realmente excepcionales. Deben ser el último recurso.
- Las excepciones mal utilizadas incrementan la complejidad
- Las excepciones reducen la encapsulación ya que las rutinas pueden requerir saber que excepciones lanza otra rutina a la que quieren llamar

Excepciones

- Siempre prefiere manejar una excepción de forma local, evita propagarlas de no ser necesario
- Evita lanzar excepciones en constructores y desctructores, a menos que los manejes de manera local. De generarse una excepción es posible que el objeto no se construya bien en memoria lo que puede causar bugs raros
- Las excepciones deben manejarse en el nivel adecuado de abstracción: si se generan en una interfaz por ejemplo, deben de ser de muy alto nivel (nivel de dominio por ejemplo)
- Al generarse la excepción debería incluirse información para saber cómo se llegó a dicha excepción (error.printStackTrace())
- No uses bloques catch vacíos

Excepciones

- Crea tus propias excepciones de dominio
- Familiarízate con las excepciones de las bibliotecas que incluyes
- A veces no deben utilizarse excepciones para manejar errores, si se puede evitar su uso mejor
- Utiliza el tipo de excepción más especifico (evita usar siempre exception)
- Para seguridad: nunca dejes visible el mensaje de trace que genera una excepción, puede dar demasiada información que un atacante puede aprovechar, utiliza logs

Determinar que tanta programación defensiva dejar en producción

- Una paradoja de la programación defensiva es que durante el desarrollo quieres que los errores sean lo más obstrusivos para que sean fáciles de notar y corregir, en cambio, en producción se desea que los errores sean lo menos obstrusivos posible, para que sea posible recuperarse de ellos y continuar trabajando
- En producción sólo deberías dejar los chequeos de errores importantes
- Remueve código que revise errores triviales
- Remueve código que resulte en una terminación del programa

Determinar que tanta programación defensiva dejar en producción

- Manda los errores a un Log para ser revisados más tarde
- Revisa que los errores que mandas sean amigables
- Deja el código que permita recuperarse de errores

Ser defensivo acerca de la programación defensiva

- Demasiada programación defensiva crea sus propios problemas
- Si revisas todos los datos posibles pasados como parámetros en todas las formas concebibles en todos los lugares concebibles, tu programa será demasiado lento
- Lo que es peor, demasiada programación defensiva crea complejidad
- Además, el código defensivo es tan propenso a tener errores como cualquier otro código (más mantenimiento)
- Piensa muy bien en los apartados en que necesitas ser defensivo y establece tus prioridades

Ejemplo

- Hacer una calculadora de línea de comandos que permita resolver operaciones como suma, resta, multiplicación y división entre 2 números
- Valida entrada
- Maneja adecuadamente excepciones
- Maneja bitácoras de errores

Ejercicio

- Hacer un programa que recibe la fecha de nacimiento de una persona en 3 entradas: día, mes, año; el programa regresa la edad de la persona
- Valida entrada
- Maneja adecuadamente excepciones
- Maneja bitácoras de errores

Ámbito

Introducción

- El ámbito se refiere a la sección de código donde algo nombrado mediante un identificador (usualmente objetos o variables) vive (puede ser recuperado)
- Cuando un ámbito desaparece todo lo que se definió dentro de ese ámbito también desaparece
- El ámbito es diferente a los modificadores de acceso
- El ámbito es diferente al tiempo de vida de un objeto
- El ámbito es algo general que aplica a cualquier identificador (clase, método, propiedad, variable), no sólo a elementos de objetos (como los modificadores) y establece no sólo acceso sino también tiempo de vida pero sólo de identificadores, no establece el tiempo de vida de objetos.

Introducción

Existen los siguientes niveles de ámbito léxico:

- Global: acceso desde cualquier punto del programa
- Objeto: acceso desde dentro de una clase
- Método: acceso desde dentro de un método
- Bloque: acceso desde dentro de un bloque de control

Ámbito global

- En Java también se le conoce como ámbito estático
- Los elementos creados en un ámbito global desaparecen hasta que el programa termina
- Las clases (a menos que se definan dentro de otra clase) tienen este ámbito
- Los elementos marcados con la palabra reservada **static** tienen un ámbito global

Ámbito global

- Cuando se tiene un elemento estático no es necesario contar con una instancia de clase para tener acceso al elemento (no se necesita de un objeto), aunque también se puede acceder al elemento desde una instancia. La buena práctica es siempre acceder a través del nombre de la clase
- Una forma de crear variables globales (compartidas en todo el programa) es definiendo dentro de alguna clase una propiedad pública estática
- CUIDADO: el uso de variables globales es una mala práctica de programación

Ámbito de Objeto

- Las propiedades y métodos se definen sintácticamente dento de una clase y viven dentro del ámbito del objeto una vez es creado
- Cuando el objeto desaparece su ámbito muere y con él todo lo que se definió en su ámbito
- Sintácticamente este ámbito comienza cuando se abren llaves después de la definición de clase y termina cuando se cierran
- Para hacer referencia explícita a este ámbito puede utilizase la palabra reservada **this**

Ámbito de método

- Sólo se pueden definir variables dentro de este ámbito
- Una vez el método termina, todos los identificadores que se definieron dentro de él también desaparecen (no así los objetos que pudieran estar referenciados por otro identificador con mayor ámbito)
- Los parámetros que recibe un método tienen ámbito de método
- Sintácticamente este ámbito comienza cuando se abren llaves después de la definición del método y termina cuando se cierran

Ámbito de bloque

- Surge cuando se utiliza alguna sentencia de control (if, while, for, etc.)
- Dentro de este ámbito sólo se pueden definir variables
- Una vez el bloque termina todos los identificadores que se definieron dentro de él desaparecen (igual que en el ámbito de método)
- Los ámbitos de bloque suelen estar demarcados por llaves, aunque existen excepciones sintácticas

Más detalles sobre ámbito

- Los ámbitos se pueden anidar. Ejemplo: dentro de un bloques se puede definir otro bloque, dentro de una clase se pueden definir métodos, etc.
- Dos identificadores no se pueden llamar igual si están en el mismo ámbito (si están en diferente ámbito si)
- Al hacer referencia a un identificador, siempre se trata de recuperar desde el ámbito más anidado primero, sino se encuentra se va al siguiente nivel de anidación

Mejores prácticas en el uso de ámbitos

- Siempre declara un identificador en el ámbito más cercano a donde será utilizado
- **NUNCA** uses variables globales, a menos que las declares como constantes, con esto evitas efectos colaterales
- Ten cuidado con el ámbito a nivel de clase/objeto cuando manejes atributos, los métodos que modifican el estado de un atributo deben declarar de forma directa esta intensión
- Un buen uso de ámbito reduce el acoplamiento y en consecuencia la complejidad

Código amigable a la recolección de basura

- La recolección de basura se refiere a la liberación automática de memoria que ya no es utilizada por el programa
- Este mecanismo automatizado lo tienen distintas plataformas de ejecución como son la máquina virtual de Java o la máquina virtual de Python
- Los lenguajes donde el programador debe manejar manualmente la memoria (como C o C++) no cuentan con este mecanismo
- El recolector de basura es un proceso especial, independiente al programa, que analiza la memoria, cuando detecta que un bloque de memoria no es referenciado por alguna referencia (variable o atributo en POO) libera esa memoria

Código amigable a la recolección de basura

- El ámbito y la recolección de basura están estrechamente relacionados
- Siempre que un ámbito termina, todas las referencias creadas en él mueren
- En consecuencia, mucha memoria puede liberarse
- Entre mayor sea el tiempo de vida de el ámbito de una referencia, menos probable es que se libere la memoria
- En ocasiones la memoria ya no se necesita pero quedó referenciada en un ámbito que sigue vivo
- La buena práctica es utilizar referencias en el ámbito con el tiempo de vida más corto posible
- Evitar usar el ámbito global también por esta razón